



Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

DAFTAR ISI

COVER.....	0
DAFTAR ISI.....	1
1. PERTEMUAN 1 - MODERN WORKFLOW: GIT & GITHUB.....	2
1.1. Tujuan Instruksi Khusus.....	2
1.2. Alat dan Bahan.....	2
1.3. Dasar Teori.....	2
1.4. Instruksi Modul.....	3
1.5. Latihan/Tugas.....	6
1.6. Sumber Bacaan.....	7
2. PERTEMUAN 2 - LARAVEL ARCHITECTURE & ROUTING.....	8
2.1. Tujuan Instruksi Khusus.....	8
2.2. Alat dan Bahan.....	8
2.3. Dasar Teori.....	8
2.4. Instruksi Modul.....	10
2.5. Latihan/Tugas.....	14
3. PERTEMUAN 3 - CONTROLLER & BLADE ENGINE.....	15
3.1. Tujuan Instruksi Khusus.....	15
3.2. Alat dan Bahan.....	15

3.3. Dasar Teori.....	15
3.4. Instruksi Modul.....	16
3.5. Latihan/Tugas.....	19
4. PERTEMUAN 4 - DATABASE: MIGRATION & SEEDING.....	20
4.1. Tujuan Instruksi Khusus.....	20
4.2. Alat dan Bahan.....	20
4.3. Dasar Teori.....	20
4.4. Instruksi Modul.....	21
4.5. Latihan/Tugas.....	28
5. PERTEMUAN 5 - ELOQUENT ORM & CRUD.....	30
5.1. Tujuan Instruksi Khusus.....	30
5.2. Alat dan Bahan.....	30
5.3. Dasar Teori.....	30
5.4. Instruksi Modul.....	31
5.5. Latihan/Tugas.....	38
6. PERTEMUAN 6 - RELATIONSHIP ELOQUENT & FILTER DATA.....	39
6.1. Tujuan Instruksi Khusus.....	39
6.2. Alat dan Bahan.....	39
6.3. Dasar Teori.....	39
6.4. Instruksi Modul.....	40

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 1 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

6.5. Latihan/Tugas.....	46
DAFTAR PUSTAKA.....	48

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 2 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

PERTEMUAN 1 - MODERN WORKFLOW: GIT & GITHUB

1.1. Tujuan Instruksi Khusus

Setelah menyelesaikan praktikum ini, praktikan diharapkan dapat:

1. Memahami konsep dasar Version Control System menggunakan Git.
2. Membuat akun dan mengelola repositori pada platform GitHub.
3. Melakukan instalasi dan konfigurasi awal Git pada komputer lokal.
4. Menerapkan alur kerja (workflow) modern menggunakan perintah git clone, git add, git commit, git push, dan git pull.

1.2. Alat dan Bahan

Alat dan bahan yang dibutuhkan pada praktikum ini sebagai berikut:

1. PC Desktop / Laptop
2. Sistem Operasi Windows 8/10/11 (atau Mac/Linux)
3. Aplikasi Git (Git Bash untuk Windows), *link download*: git-scm.com
4. Visual Studio Code, *link download*: code.visualstudio.com
5. Web Browser (Chrome, Firefox, Edge, dll)
6. Koneksi Internet aktif
7. Akun Email aktif untuk pendaftaran akun GitHub

1.3. Dasar Teori

1.3.1. Tentang Git dan GitHub

Git adalah *Version Control System* (VCS) terdistribusi yang digunakan untuk melacak perubahan pada file proyek dan berkolaborasi dengan banyak orang. Git bekerja secara lokal di komputer pengembang.

GitHub adalah layanan *cloud hosting* berbasis web untuk menyimpan repositori Git. Jika Git adalah mesinnya, maka GitHub adalah tempat kita menyimpan dan membagikan kode (*source code*) kita agar bisa diakses secara publik maupun privat, serta berkolaborasi dengan tim (*collaborators*).

1.3.2. Siklus Kerja Dasar Git (Git Workflow)

melibatkan beberapa perintah utama berikut:

1. **git clone**: Menggandakan (menyalin) repositori yang ada di GitHub ke dalam komputer lokal.
2. **git add**: Menambahkan file yang baru dibuat atau diubah ke dalam *staging area* (area persiapan sebelum disimpan permanen).
3. **git commit**: Menyimpan riwayat perubahan dari *staging area* secara permanen ke dalam Git lokal, dilengkapi dengan pesan (*commit message*).
4. **git push**: Mengunggah *commit* yang ada di komputer lokal ke server GitHub.
5. **git pull**: Mengambil (*download*) perubahan terbaru dari server GitHub ke komputer lokal, berguna ketika bekerja dalam tim dan ada anggota tim lain yang memperbarui kode.

Dan masih ada banyak perintah lagi yang dapat anda pelajari di dokumentasi resminya di <https://git-scm.com/docs>

1.4. Instruksi Modul

1.4.1. Membuat Akun GitHub

1. Buka *web browser* dan kunjungi alamat github.com.
2. Klik tombol **Sign up** di pojok kanan atas.
3. Masukkan alamat email aktif Anda, buat *password* yang kuat, dan tentukan *username*.
4. Lakukan verifikasi teka-teki visual (*captcha*) sesuai instruksi yang diminta (misal: memutar gambar hingga posisinya lurus).
5. Buka kotak masuk email Anda, cari email dari GitHub, lalu klik tombol **Verify email address** atau masukkan kode OTP yang diberikan.
6. Login kembali ke GitHub.

1.4.2. Membuat Repositori Baru

1. Pada halaman utama GitHub, klik ikon + di sudut kanan atas dan pilih New

repository (atau klik tombol hijau Create repository).

2. Isi form pembuatan repositori sebagai berikut:

- a. **Repository name:** `personal-goal-belajar-git` (atau nama lain tanpa spasi).
- b. **Description:** *Repositori untuk menyimpan rencana dan cita-cita ke depan.*
- c. Pilih **Public** agar repositori bisa dilihat oleh orang lain.
- d. Centang bagian **Add a README file** agar repositori Anda tidak kosong saat pertama kali dibuat.

3. Klik tombol hijau **Create repository**.

4. Repositori Anda telah berhasil dibuat. Salin URL repositori tersebut (contoh:

`https://github.com/username/personal-goal-belajar-git.git`) dari tombol **Code**.

1.4.3. Melakukan Clone Repositori ke Lokal

1. Buat sebuah folder baru di komputer Anda (misal: di drive `D:\PraktikumDB\XXXX`).
2. Buka folder tersebut, klik kanan pada area kosong, lalu pilih **Open Git Bash here** (atau buka terminal melalui Visual Studio Code dan arahkan ke folder tersebut).
3. Ketikkan perintah berikut untuk mengunduh repositori ke komputer lokal Anda (ganti URL dengan URL repositori Anda masing-masing):

Shell

```
git clone https://github.com/username/personal-goal-belajar-git.git
```

4. Masuk ke dalam folder yang baru saja di-clone:

1.4.4. Coba Git Add (Menambahkan File Baru)

1. Buka folder **personal-goal-belajar-git** tersebut menggunakan Visual Studio Code.
2. Buatlah satu file baru bernama **personal-goal.txt**
3. Isi file tersebut dengan daftar goal, target, atau mimpi Anda di tahun ini (contoh:
1. Konsisten masuk kuliah tanpa bolos; 2. Mendapatkan nilai A di semua makul;). Simpan file tersebut (**Ctrl+S**).
4. Masuk ke terminal VS Code
5. Ketikkan perintah berikut untuk memasukkan file ke dalam *staging area*:

Shell

```
git add personal-goal.txt
```

(Tips: Jika ingin menambahkan semua file sekaligus, gunakan perintah **git add .**)

1.4.5. Melakukan Git Commit

1. Setelah file berada di *staging area*, kita perlu merekam perubahan tersebut secara permanen di lokal dengan menyertakan pesan *commit*.
2. Ketikkan perintah berikut di terminal:

Shell

```
git commit -m "menambahkan file personal-goal.txt"
```

3. Tekan *Enter*. Akan muncul informasi bahwa file berhasil di-*commit* (tercatat *1 file changed, insertions*).

1.4.6. Melakukan Git Push

1. Perubahan saat ini baru tersimpan di komputer lokal. Untuk mengunggahnya ke GitHub, gunakan perintah:

Shell

```
git push origin main
```

2. Catatan: Jika diminta login, masukkan kredensial GitHub Anda atau gunakan otentikasi browser
3. Buka halaman repositori GitHub Anda di *browser* lalu *refresh*. File `personal-goal.txt` kini sudah muncul di sana.

1.4.7. Melakukan Git Pull

1. Untuk mensimulasikan `git pull`, kita akan mengubah file langsung dari GitHub.
2. Di halaman repositori GitHub Anda, klik file `personal-goal.txt`, lalu klik ikon pensil (**Edit this file**).
3. Tambahkan satu baris resolusi baru, lalu gulir ke bawah dan klik tombol hijau **Commit changes...** Tulis pesan *commit* dan simpan.
4. Sekarang, file di GitHub lebih *up-to-date* dibandingkan yang ada di komputer lokal Anda.
5. Kembali ke terminal Visual Studio Code di komputer Anda, dan ketikkan perintah:

Shell

```
git pull origin main
```

6. Git akan mengunduh perubahan dari GitHub. Buka file `personal-goal.txt` di VS Code, dan Anda akan melihat baris baru yang ditambahkan dari GitHub tadi sudah muncul secara otomatis.

1.5. Latihan/Tugas

1. Buatlah sebuah repositori baru secara publik di GitHub dengan format nama `[NIM]_TugasGit_DB`.
2. Lakukan proses **Clone** ke komputer lokal Anda.

3. Buat sebuah file `index.html` sederhana yang berisi biodata diri (Nama, NIM, Program Studi).
4. Lakukan proses **Add**, **Commit**, dan **Push** file tersebut ke repositori Anda.
5. Lakukan edit file `index.html` secara langsung melalui antarmuka GitHub (tambahkan Hobi Anda), lakukan *commit* di sana, lalu lakukan **Pull** ke komputer lokal Anda.
6. Buatlah laporan praktikum yang berisi tangkapan layar (*screenshot*) histori *commit* dari repositori GitHub Anda yang membuktikan seluruh proses di atas telah dilakukan! Jelaskan fungsi dari masing-masing perintah yang digunakan

1.6. Sumber Bacaan

- [1] Galih, Sandhika. (2019). #2 *BEKERJA DENGAN GITHUB*.. Diakses dari <https://www.youtube.com/watch?v=Q3Id0DgcrXY>
- [2] Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. (Dapat diakses melalui <https://git-scm.com/book/en/v2>)
- [3] GitHub Docs. (2024). *Hello World - GitHub Docs*. Diakses dari <https://docs.github.com/en/get-started/quickstart/hello-world>

PERTEMUAN 2 - LARAVEL ARCHITECTURE & ROUTING

2.1. Tujuan Instruksi Khusus

Setelah menyelesaikan praktikum ini, praktikan diharapkan dapat:

1. Memahami arsitektur dasar dan struktur folder pada framework Laravel.
2. Melakukan instalasi environment lokal melalui Laragon (yang sudah memuat PHP, web server, database, dan Composer).
3. Melakukan instalasi framework Laravel menggunakan Composer.
4. Mengerti konsep dan cara kerja Routing di dalam Laravel ([routes/web.php](#)).
5. Membuat rute dasar untuk menampilkan halaman statis dan sistem navigasi antar halaman.
6. Memahami dan menerapkan styling antarmuka sederhana dengan Tailwind CSS.

2.2. Alat dan Bahan

Alat dan bahan yang dibutuhkan pada praktikum ini sebagai berikut:

1. PC Desktop / Laptop
2. Sistem Operasi Windows 8/10/11
3. Aplikasi **Laragon Full** (sudah mencakup Apache, MySQL, PHP, dan Composer bawaan), link unduh: laragon.org/download
4. Visual Studio Code, link download: code.visualstudio.com
5. Web Browser (Chrome, Firefox, Edge, dll)
6. Koneksi Internet aktif

2.3. Dasar Teori

2.3.1. Tentang Laravel dan Arsitektur MVC

Laravel adalah framework aplikasi web berbasis PHP yang ekspresif dan elegan, dirancang untuk memudahkan proses pengembangan web secara cepat (Rapid Application Development). Laravel menggunakan pola arsitektur **Model-View-Controller (MVC)**:

- **Model:** Representasi data dan interaksi dengan database (menggunakan Eloquent ORM).

- **View:** Tampilan antarmuka pengguna (User Interface), biasanya menggunakan Blade template engine.
- **Controller:** Mengatur lalu lintas data (logika bisnis) antara Model dan View.

2.3.2. Struktur Folder Laravel

Beberapa folder penting dalam Laravel yang harus diperhatikan:

- **app/:** Menyimpan inti kode aplikasi (seperti Models dan Controllers).
- **public/:** Menyimpan file yang dapat diakses langsung oleh browser (CSS, JS, Gambar) serta file **index.php** sebagai titik masuk utama.
- **resources/:** Menyimpan file raw yang belum di-compile dan file tampilan (Views).
- **routes/:** Menyimpan definisi rute (URL) aplikasi. File **web.php** digunakan untuk rute web (antarmuka browser).

2.3.3. Routing Dasar

Routing merupakan mekanisme untuk mencocokkan URL di browser dengan sebuah tindakan yang harus dilakukan oleh aplikasi. Di Laravel, semua request dari web browser dicatat dan diproses melalui routing sehingga navigasi di aplikasi menjadi lebih rapi.

2.3.4. Pengenalan Tailwind CSS

Tailwind CSS adalah framework CSS *utility-first* yang sangat populer di industri modern web development karena kecepatan dan fleksibilitasnya. Alih-alih menulis aturan CSS di file terpisah dan membuat penamaan class satu per satu, kita dapat langsung menambahkan class *utility* bawaan Tailwind ke elemen HTML. Untuk tahap pembelajaran awal di Laravel, Tailwind dapat disisipkan secara instan dan cepat menggunakan metode CDN di file View.

Beberapa contoh kategori *utility class* dasar yang sering digunakan pada Tailwind CSS:

- **Spasi (Padding & Margin):** **p-4** (padding 1rem), **m-2** (margin 0.5rem), **px-4** (padding horizontal), **mt-4** (margin atas).
- **Tipografi:** **text-center** (teks rata tengah), **text-xl** (ukuran teks besar), **font-bold** (teks tebal), **text-blue-600** (warna teks).

- **Background & Border:** `bg-slate-100` (warna latar), `border border-gray-200` (garis tepi), `rounded-lg` (sudut membulat).
- **Layouting:** `flex` (flexbox), `justify-center` (rata tengah horizontal), `items-center` (rata tengah vertikal).

2.4. Instruksi Modul

2.4.1. Instalasi Lingkungan Laragon (termasuk Composer)

1. Buka web browser dan kunjungi laragon.org/download.
2. Unduh versi **Laragon Full** (direkomendasikan untuk pengguna Windows) dan jalankan file installer tersebut mengikuti pengaturan *default* hingga instalasi selesai.
3. Setelah selesai, buka aplikasi Laragon lalu klik tombol **Start All** untuk menyalakan servis web (Apache) dan *database* (MySQL).
4. Klik opsi **Terminal** di antarmuka Laragon (atau klik kanan area kosong Laragon -> Laragon Terminal) untuk membuka jendela komando (Command Prompt khusus).
5. Pada terminal Laragon yang terbuka, ketik `composer -v` lalu tekan Enter. Jika berhasil, akan muncul logo *text-art* bertuliskan Composer yang berarti *dependency manager* bawaan Laragon Anda siap digunakan.

2.4.2. Instalasi Framework Laravel

1. Tetap buka jendela Terminal bawaan Laragon Anda. Biasanya terminal ini secara otomatis akan mengarahkan ke folder kerja Laragon (`C:\laragon\www\`).
2. Jalankan perintah instalasi Laravel (kita asumsikan nama project yang akan diinstal adalah `laravel-app`):

Shell

```
composer create-project laravel/laravel laravel-app
```

3. Tunggu hingga proses *download* dependency paket dan scaffolding selesai.
4. Masuk ke dalam direktori project yang baru saja dibuat dengan perintah: `cd laravel-app`.

2.4.3. Menjalankan Server Development Lokal

1. Pastikan Anda masih berada di dalam direktori `laravel-app` pada Terminal.
2. Jalankan server lokal bawaan Laravel dengan perintah:

Shell

```
php artisan serve
```

3. Buka web browser dan akses URL `http://127.0.0.1:8000` atau `http://localhost:8000`. Jika muncul halaman depan default Laravel, maka instalasi sukses!

2.4.4. Menambahkan Route Dasar

1. Buka folder `laravel-app` menggunakan Visual Studio Code.
2. Buka file `routes/web.php`.
3. Tambahkan rute baru untuk membuat halaman pendahuluan:

PHP

```
Route::get('/tentang', function () {  
  
    return '<h1>Ini adalah Halaman Tentang Aplikasi Event Hub</h1>';  
  
});
```

4. Buka browser dan akses alamat `http://localhost:8000/tentang`. Validasi pesan tampil pada layar browser.

2.4.5. Membuat Route yang Menghubungkan ke View

1. Di Visual Studio Code, arahkan ke direktori `resources/views/`.
2. Buat file baru dengan nama `contact.blade.php`.
3. Isi dengan HTML sederhana yang memuat sebuah link yang kembali ke Home (`/`): Modul

```
HTML
<!DOCTYPE html>

<html>

<head><title>Halaman Kontak</title></head>

<body>

    <h1>Hubungi Kami</h1>

    <p>Email: admin@amikomeventhub.com</p>

    <a href="/">Kembali ke Home</a>

</body>

</html>
```

4. Buka kembali file `routes/web.php` dan tambahkan rute untuk View tersebut:

```
PHP
Route::get('/kontak', function () {

    return view('contact');

});
```

5. Akses `http://localhost:8000/kontak` di web browser, lalu klik Link 'Kembali ke Home' untuk mencoba rutenya berjalan.

2.4.6. Menerapkan Styling dengan Tailwind CSS (via CDN)

1. Buka kembali file `resources/views/contact.blade.php`. 2. Tambahkan script CDN Tailwind di dalam tag `<head>` dan cobalah berikan *styling* yang lebih modern menggunakan class bawaan Tailwind:

```
HTML
<!DOCTYPE html>

<html>

    <head>

        <title>Halaman Kontak</title>
        Universitas AMIKOM Yogyakarta
        Fakultas Ilmu Komputer - Program Studi S1 Sistem
        Informasi

        <!-- Menyisipkan Tailwind CSS melalui CDN -->

        <script src="https://cdn.tailwindcss.com"></script>

    </head>

    <body class="bg-slate-100 flex items-center justify-center h-screen">

        <div class="bg-white p-8 rounded-xl shadow-lg border border-slate-200
        text-center max-w-sm w-full">

            <h1 class="text-2xl font-bold text-slate-800 mb-2">Hubungi Kami</h1>

            <p class="text-slate-500 mb-6">Email: admin@amikomeventhub.com</p>

            <a href="/" class="inline-block bg-indigo-600 text-white font-semibold
            py-2 px-6 rounded-lg hover:bg-indigo-700 hover:shadow-md transition
            duration-300">

                Kembali ke Home

            </a>

        </div>

    </body>

</html>
```

3. Silakan muat ulang (refresh) halaman <http://localhost:8000/kontak> di browser Anda. Tampilan halaman tersebut kini akan berubah menjadi jauh lebih rapi, terpusat di tengah layar, dan memiliki gaya yang kekinian.

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 14 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

2.5. Latihan/Tugas

1. Lakukan instalasi proyek Laravel baru sesuai langkah di instruksi modul (Gunakan nama folder project: [\[NIM\]_LaravelApp](#)).
2. Buat tiga rute ([Route::get](#)) tambahan untuk halaman statis berikut:
 - [/profil](#): Menampilkan profil praktikan.
 - [/katalog](#): Menampilkan placeholder daftar event AmikomEventHub.
 - [/bantuan](#): Menampilkan FAQ sederhana.
3. Buat file Blade Template View di folder [resources/views/](#) untuk masing-masing rute tersebut ([profil.blade.php](#), [katalog.blade.php](#), [bantuan.blade.php](#)).
4. Gunakan dan padukan kelas-kelas **Tailwind CSS via CDN** untuk menghias masing-masing halaman View (Profil, Katalog, Bantuan) agar terlihat lebih berstruktur dan indah (berikan warna latar belakang modern, tipografi yang pas, padding, border radius, dan efek seperti *hover* atau elemen *card*).
5. Pastikan di setiap halaman terdapat link HyperText ([<a>](#)) yang di-styling menyerupai tombol agar pengguna dapat saling berpindah melintasi layar Home, Profil, Katalog, Bantuan, dan Kontak dengan mudah.
6. Screenshot file [web.php](#), screenshot isi direktori [resources/views/](#), serta screenshot pengujian desain tampilan (UI) di browser (navigasi antar halaman). 7. Susun semua screenshot beserta sedikit penjelasan langkah kerja dalam format PDF dan kumpulkan di submission tugas Anda.

PERTEMUAN 3 - CONTROLLER & BLADE ENGINE

3.1. Tujuan Instruksi Khusus

Setelah menyelesaikan praktikum ini, praktikan diharapkan dapat:

- Memahami fungsi dan cara kerja **Controller** pada framework Laravel.
- Memahami keunggulan sistem **Blade Templating Engine**.
- Menggunakan fitur ekstensibilitas Blade seperti `@yield`, `@extends`, `@section`, dan `@include`.
- Memecah struktur template HTML utuh (prototype) menjadi komponen layout yang *reusable* untuk sisi Pengguna (User) dan Admin.
- Menghubungkan View dengan Route melalui Controller tanpa logika bisnis (setup antarmuka aplikasi secara menyeluruh).

3.2. Alat dan Bahan

Alat dan bahan yang dibutuhkan pada praktikum ini sebagai berikut:

1. PC Desktop / Laptop
2. Aplikasi XAMPP / Laragon
3. Composer
4. File *template prototype HTML* lengkap (index, event-detail, checkout, ticket, admin-dashboard, admin-events, admin-transactions). Dapat di download melalui link berikut s.amikom.ac.id/template-db-si
5. Visual Studio Code & Web Browser

3.3. Dasar Teori

3.3.1. Controller dalam MVC

Controller merupakan bagian dari MVC yang bertugas menerima sebuah instruksi (HTTP Request), memproses logika sistem, dan mengembalikan Response (biasanya berupa pemanggilan View). Penggunaan Controller membuat kode lebih tertata karena pemanggilan logic tidak lagi ditumpuk pada file `routes/web.php`. Cara membuatnya: `php artisan make:controller NamaController`

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 16 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

3.3.2. Blade Templating Engine

Blade adalah *templating engine* andalan Laravel. Blade memperbolehkan kita memecah visual UI yang redundan (seperti navigasi dan footer) menjadi layout terpusat yang bisa dipakai berulang kali. Fitur utama Blade:

- `@yield('content')`: Lubang kosong / penanda pada Master Layout yang nantinya diisi oleh view spesifik.
- `@extends('layouts.app')`: Perintah pada file view untuk menggunakan sebuah rancangan (master layout).
- `@section('content') ... @endsection`: Blok kode HTML untuk mengisi `@yield`.

3.4. Instruksi Modul

3.4.1. Persiapan File Prototype

Pindahkan atribut frontend seperti folder `assets/` (berisi gambar, icon, dsb.) dari Prototype ke dalam folder `public/` agar bisa diakses oleh web browser secara langsung.

3.4.2. Membuat Master Layout User (`layouts.app`)

1. Buat direktori `resources/views/layouts/`.
2. Buat file `app.blade.php`.
3. Pindahkan struktur HTML kerangka utama dari file `index.html` (Mulai dari tag `<html>`, `<head>`, Navbar, hingga `<footer>` dan `</html>`) ke dalam file ini.
- 4.

Sisipkan tag `@yield('content')` di mana konten utama halaman (kolom tengah aplikasi) akan ditampilkan.

3.4.3. Integrasi Halaman Sisi Pengguna (User)

1. Buat Controller bernama `HomeController`. Cara membuatnya: `php artisan make:controller HomeController`
2. Gunakan method `index()` untuk mengembalikan `view('welcome')`. Modul

Praktikum : [Digital Bisnis [SI148]] Hal. 17 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

3. Buat file `resources/views/welcome.blade.php`. Panggil Master Layout dengan `@extends('layouts.app')` dan isi kontennya dengan Hero Section & Event Grid dari prototype, lalu letakkan di dalam `@section('content')`.
4. Buat file `resources/views/event-detail.blade.php`. Pastikan ia me-`@extends('layouts.app')`. Copy bagian konten detail acara dari prototype `event-detail.html`.
5. Buat file `resources/views/checkout.blade.php` beserta struktur template dari `checkout.html`.
6. Buat file `resources/views/ticket.blade.php` beserta isi desain dari `ticket.html`.
7. Buat `EventController` dan arahkan method yang sesuai (misal: `show()`) untuk merender halaman `event-detail` dan `checkout`. Serta `TicketController` atau pada spesifik method untuk merender `ticket`. *Perhatikan:* Jangan pusingkan logika databasenya dulu, fokus hanya pada pemanggilan View.
8. Cara membuatnya: `php artisan make:controller EventController`

3.4.4. Membuat Master Layout Admin (`layouts.admin`)

1. Di dalam folder `resources/views/layouts/`, buat file master baru bernama

`admin.blade.php`.

2. Buka prototype `admin-dashboard.html`. Ambil struktur dasar panel antarmuka Admin (Side menu bar, top navbar header), dan letakkan dalam `admin.blade.php`.
3. Sisipkan `@yield('content')` pada area grid container sebelah kanan tempat tabel dan chart data akan muncul.

3.4.5. Integrasi Halaman Sisi Admin (Admin Panel)

1. Buat folder `resources/views/admin/`.

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 18 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

2. Buat tiga file view: `dashboard.blade.php`, `events.blade.php`, dan `transactions.blade.php`. Semuanya menggunakan `@extends('layouts.admin')`.
3. Salin sisa kode bagian kolom tabel atau chart dari masing-masing prototype ke `@section('content')` file bersangkutan.
4. Buat sebuah Controller khusus Admin melalui terminal: `php artisan make:controller Admin/DashboardController` dan `Admin/EventController`.
5. Arahkan untuk menampilkan (return view) dashboard dan events.

3.4.6. Mendaftarkan Seluruh Routing di `web.php`

Buka file `routes/web.php` dan buatlah rute GET untuk menghubungkan semua navigasi kita menuju fungsi-fungsi di Controller yang baru Anda rintis di atas. Misalnya:

```
PHP
use App\Http\Controllers\HomeController;

use App\Http\Controllers\EventController;

use App\Http\Controllers\Admin\DashboardController;

// Rute User Area
```

```

Route::get('/', [HomeController::class, 'index'])->name('home');

Route::get('/event/1', [EventController::class, 'show'])->name('events.show');

Route::get('/checkout', [EventController::class, 'checkout'])->name('checkout');

Route::get('/my-ticket', [EventController::class, 'ticket'])->name('ticket');

// Rute Admin Area

Route::group(['prefix' => 'admin', 'as' => 'admin.'], function () { Modul Praktikum

```

: [Digital Bisnis [SI148]] Hal. 19 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

Route::get('/', [DashboardController::class, 'index'])->name('dashboard');

Route::get('/events', [EventController::class, 'indexAdmin'])->name('events.index');

// dan seterusnya...

});

```

Lakukan pengujian aplikasi dengan pindah secara manual menggunakan URL di web browser lokal. Pastikan semua desain muncul tanpa masalah.

3.5. Latihan/Tugas

Sebagai tahapan untuk melengkapi seluruh panel Manajemen Administrator, cobalah mendesain dan menyiapkan pondasi untuk fitur Manajemen Kategori. Instruksinya:

1. Buat file tampilan baru:

`resources/views/admin/categories/index.blade.php`. 2. Gunakan `@extends('layouts.admin')`.

3. Desain (HTML/Tailwind) atau modifikasi tabel dari tampilan Admin Events untuk

membuat *Graphical User Interface* sederhana yang dapat digunakan Admin untuk melihat kumpulan teks Kategori (misal: "Seminar", "Konser", "Workshop"), sekaligus tambahkan tombol UI "Tambah", "Edit", dan "Hapus". (Belum memerlukan fungsi database sungguhan).

4. Buat rute baru: `Route::get('/admin/categories')` yang terhubung ke sebuah controller Kategori.
5. Screenshot struktur folder Anda beserta hasil tampilan halamannya di browser, dan tulis penjelasan singkat dalam file PDF sebagai laporan Anda kepada asisten.

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 20 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

PERTEMUAN 4 - DATABASE: MIGRATION & SEEDING

4.1. Tujuan Instruksi Khusus

Setelah menyelesaikan praktikum ini, praktikan diharapkan dapat:

- Memahami konsep Migration pada Laravel sebagai *version control* untuk database.
- Membuat struktur tabel database melalui file Migration tanpa menulis *query* SQL secara manual.
- Memahami konsep Seeding dan Factory untuk mengisi tabel dengan data awal (*dummy data*).
- Mengimplementasikan pembentukan relasi dan *foreign key* di dalam formasi tabel Migration.

4.2. Alat dan Bahan

Alat dan bahan yang dibutuhkan pada praktikum ini sebagai berikut:

1. PC Desktop / Laptop
2. Sistem Operasi Windows 8/10/11 (atau Mac/Linux)
3. Aplikasi XAMPP / Laragon (Web Server & Database engine berjalan)
4. Visual Studio Code

5. Web Browser
6. Terminal / Git Bash

4.3. Dasar Teori

4.3.1. Pengenalan Migration

Migration adalah fitur dalam Laravel yang memungkinkan kita untuk mengelola struktur database secara terprogram menggunakan PHP. Hal ini membuat tim *developer* mudah untuk berbagi skema database dan menjaga konsistensi kolom di antara berbagai lingkungan (development, staging, production) tanpa harus mengekspor/mengimpor file **.sql**.

Saat menjalankan perintah *artisan* untuk membuat migration, Laravel akan *men-generate* sebuah file di folder **database/migrations/** dengan *timestamp*. File ini memiliki fungsi

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 21 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

up() untuk membuat atau memodifikasi tabel, dan fungsi **down()** untuk menghapus atau mengembalikan modifikasi tersebut (Reverse/Rollback).

4.3.2. Menyambungkan Aplikasi dengan Database (File **.env**)

Sebelum *merancang* database via Migration, Laravel harus terhubung dengan database lokal kita (misalnya MySQL atau MariaDB bawaan Laragon). Pengaturan konfigurasi koneksi ini tersimpan pada file **.env**. Kita cukup mengubah parameter **DB_DATABASE**, **DB_USERNAME**, dan **DB_PASSWORD**.

4.3.3. Database Seeding & Faker

Seeding adalah proses pengisian data awal secara otomatis ke dalam database ketika tahap pengembangan. Fitur ini sangat berguna agar kita tidak perlu memasukkan data spesifik uji coba berulang kali secara manual dari *phpMyAdmin*.

Untuk menghasilkan baris data palsu dalam jumlah puluhan hingga ribuan (seperti nama, alamat, email, dll yang *random* namun masuk akal), PHP menyediakan sebuah *library* yang dinamakan **Faker**. Dalam Laravel, seringkali Seeding dipadukan dengan Factory dan Faker. Namun, untuk master data seperti akun admin panel, umumnya didefinisikan secara konstan di dalam method **run()** di Class Seeder.

4.4. Instruksi Modul

4.4.1. Konfigurasi Database (.env)

Pada praktikum kali ini, kita akan membuat rancangan database untuk menyokong aplikasi **AmikomEventHub**.

1. Aktifkan XAMPP / Laragon (Apache dan MySQL/MariaDB).
2. Buka dan login ke phpMyAdmin , lalu buat sebuah *database* baru dan beri nama, seperti **eventtiket_db** (atau gunakan NIM pada penamaannya).
3. Buka file **.env** di dalam root direktori proyek **laravel-app**
- Anda. 4. Sesuaikan bagian blok konfigurasi database Anda:

```
None
DB_CONNECTION=mysql
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 22 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=eventtiket_db
DB_USERNAME=root
DB_PASSWORD=
```

*(Kosongkan teks password jika Anda menggunakan pengaturan bawaan (default) XAMPP Windows. Beri value berupa **"root"** pada password bila Anda menggunakan MAMP/Laragon).*

4.4.2. Membuat Migration Baru

Merujuk pada PRD rancangan produk, kita membutuhkan tiga tabel operasional tambahan di luar tabel dasar yang ada:

- **users** (Digunakan secara autentik untuk Admin Penyelenggara)
- **categories** (Kategori daftar Event)

- **events** (Data penawaran acara komersil seperti seminar/konser)
- **transactions** (Pencatatan pemesanan dan checkout dari Customer)

Jalankan perintah berikut di Terminal IDE VS Code Anda secara berurutan untuk men-*generate* file model dan strukturnya sekaligus:

```
Shell
php artisan make:model Category -m

php artisan make:model Event -m

php artisan make:model Transaction -m
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 23 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

*(Parameter **-m** akan menginstruksikan barisan agar otomatis mendesain file Migration pembentukan tabel bersamanya. Tabel **users** diabaikan karena by default sudah ada).*

4.4.3. Merancang Skema Database

Silakan lihat tab folder dan buka kumpulan file migration bawaan yang ada di **database/migrations/**. Ubah method **up()** pada setiap file *Generate* terbaru untuk menyesuaikannya dengan ERD dan PRD proyek kita.

1. Tabel Users (Opsional) *Abaikan file init jika Anda sudah menggunakan sistem modern Laravel 11+, namun pastikan menambahkan kolom role jika belum.* Sebagai contoh pada file *create_users_table*:
Anda bisa sisipkan
`$table->string('role')->default('user');` setelah field
`$table->string('password');` (Atau sisipkan pada seeder).

2. Tabel Categories (Buka file ...create_categories_table.php) Ketikkan perancangan berikut:

```
PHP
public function up(): void
```

```

{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();

        $table->string('name');

        $table->string('slug')->unique();

        $table->timestamps();
    });
}

```

3. Tabel Events (Buka file ...create_events_table.php) Disini kita akan membuat keterhubungan identitas *Foreign Key* dari kategori menggunakan fitur **constrained**.

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 24 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

PHP

```

public function up(): void
{
    Schema::create('events', function (Blueprint $table) {
        $table->id();

        // Foreign Key relasi one-to-many ke categories

        $table->foreignId('category_id')->constrained()->cascadeOnDelete();

        $table->string('title');

        $table->text('description')->nullable();

        $table->dateTime('date');

        $table->string('location');

        $table->integer('price');

        $table->integer('stock');

        $table->string('poster_path')->nullable();
    });
}

```

```

        $table->timestamps();

    });

}

```

4. Tabel Transactions (Buka file ...create_transactions_table.php) Tabel transaction digunakan dalam pengadministrasian *guest checkout*.

```

PHP
public function up(): void
{
    Schema::create('transactions', function (Blueprint $table) {

        $table->id();

```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 25 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

        $table->foreignId('event_id')->constrained()->cascadeOnDelete();

        $table->string('order_id')->unique(); // No Pesanan unik

        $table->string('customer_name');

        $table->string('customer_email');

        $table->string('customer_phone');

        $table->integer('total_price');

        $table->string('status')->default('Pending');

        $table->string('snap_token')->nullable();

        $table->timestamps();

    });

}

```

4.4.4. Mengeksekusi (Run) Migrate

Pada dasarnya file-file migration hanyalah pendeskripsi struktur tabel. Setelah dirasa benar, kita harus mentranslasikannya ke Database Server dengan baris perintah artisan ini:

```
Shell  
php artisan migrate
```

*Trik: Bila melakukan kesalahan tipikal nama field, Anda bisa memodifikasi script migrasi ulang dengan segera dan menggunakan **php artisan migrate:fresh** untuk menjalankan ulang migrasi secara fresh (menghapus struktur tabel yang telah ada di database sebelumnya dan menimpa dengan struktur migrasi baru)*

4.4.5. Mengisi Data Awal melalui Seeder

Sekarang, mari jadwalkan pendataan otomatis supaya tampilan kita di pertemuan selanjutnya tidak kosong melalui **Seeder**. Buka dan modifikasilah file utama **database/seeder/DatabaseSeeder.php** sebagai berikut:

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 26 dari 67
{

```
public function run(): void  
{  
  
    // 1. Akun Admin Utama  
  
    \App\Models\User::create([  
  
        'name' => 'Admin Amikom',  
        'Universitas AMIKOM Yogyakarta'  
    ]);  
  
    namespace Database\Seeders;  
  
    use Illuminate\Database\Seeder;  
  
    class DatabaseSeeder extends Seeder
```

Fakultas Ilmu Komputer - Program Studi S1 Sistem
Informasi

```
'email' => 'admin@amikom.ac.id',  
'password' => bcrypt('password'),  
'role' => 'admin',  
]);  
  
// 2. Insert Kategori Event  
  
$category = \App\Models\Category::create([  
    'name' => 'Seminar IT',  
    'slug' => 'seminar-it',  
]);
```

```

$category2 = \App\Models\Category::firstOrCreate([

    'name' => 'Entertainment',

    'slug' => 'entertainment',

]);

// 3. Insert Sampel Events

\App\Models\Event::create([

    'category_id' => $category2->id,

    'title' => 'Jazz Night 2025',

    'description' => 'Nikmati malam yang indah dengan alunan musik jazz
yang merdu.',

    'date' => '2026-05-10 19:00:00',

    'location' => 'Amikom Baru',

    'price' => 50000,

    'stock' => 100,

    'poster_path' => 'posters/event-1.png',

]);

\App\Models\Event::create([

    'category_id' => $category->id,

    'title' => 'Hackaton - Unleash Your Inner Developer',

    'description' => 'Ayo asah skill coding kamu dan ciptakan solusi
inovatif untuk tantangan masa depan!',

    'date' => '2026-05-05 10:00:00',

```

```

        'location' => 'Inkubator Amikom',

        'price' => 50000,

        'stock' => 100,

        'poster_path' => 'posters/event-2.png',

    ]);

    \App\Models\Event::create([

        'category_id' => $category->id,

        'title' => 'AI & FUTURE TECH SUMMIT 2026',

        'description' => 'Jelajahi tren terkini dalam kecerdasan buatan dan
teknologi masa depan bersama para ahli di bidangnya.',

        'date' => '2026-05-01 13:00:00',

        'location' => 'Cinema Unit 6',

        'price' => 50000,

        'stock' => 100,

        'poster_path' => 'posters/event-3.png',

    ]);

}

}

```

Akhiri langkah pengisian dengan baris perintah yang mengeksekusinya:

```

Shell
php artisan db:seed

```

Silakan Anda verifikasi menggunakan phpMyAdmin, jika perintah terekstrak rapi, maka tabel `categories` dan `events` sudah muncul berisikan data tersebut.

4.5. Latihan/Tugas

1. Lakukan instalasi database dan konfigurasi migration pada proyek *local* Anda dengan panduan persis di atas.
2. Bereksperimenlah di bagian `DatabaseSeeder.php` untuk menanamkan minimal 3 kategori dan 6 jenis kegiatan/event acak yang logis dan bervariasi (*contoh: UI/UX Masterclass, E-Sport U-Champ, dst*).
3. Ambil screenshot penunjukkan *log run* `php artisan migrate:fresh --seed`.
Jika berhasil, simpan screenshot tabel dari phpMyAdmin (*bisa events atau users*) dan kumpulkan laporannya dengan langkah singkat di submission.

PERTEMUAN 5 - ELOQUENT ORM & CRUD

5.1. Tujuan Instruksi Khusus

Setelah menyelesaikan praktikum ini, praktikan diharapkan dapat:

- Memahami konsep Object-Relational Mapping (ORM) menggunakan *Eloquent* di Laravel.
- Membuat fitur manajemen data (*Create, Read, Update, Delete*) secara utuh (mencakup *Controller* dan antarmuka *View*) pada sisi *Admin*.
- Memanfaatkan sistem validasi sederhana pada pengolahan *Request*.
- Memahami fungsi dan efisiensi *Route Resource* pada ekosistem URL Laravel.

5.2. Alat dan Bahan

Alat dan bahan yang dibutuhkan pada praktikum ini sebagai berikut:

1. PC Desktop / Laptop
2. Sistem Operasi Windows/Mac/Linux
3. XAMPP / Laragon (Pastikan Web Server & Database engine berjalan)
4. Visual Studio Code & Web Browser
5. Terdapat sebuah proyek **laravel-app** yang sudah berisi integrasi template Blade (diakses pada Admin) dan struktur Database (*Migration & Seeding*) yang berhasil dilewati dari Pertemuan 3 & 4.

5.3. Dasar Teori

5.3.1. Eloquent ORM

Eloquent adalah *Object-Relational Mapper* (ORM) bawaan Laravel yang sangat efisien dan ekspresif. Setiap tabel pada basis data direpresentasikan melalui sebuah "Model" (misalnya: tabel **events** diwakili oleh Model **Event.php**). Dengan sistem ini, kita berinteraksi dengan basis data tanpa menggunakan kueri SQL murni, melainkan menggunakan metode bawaan Model seperti **Event::all()**, **Event::create()**, dan **\$event->delete()**.

5.3.2. Operasi CRUD

CRUD merupakan basis interaksi standar untuk sebuah tabel yang terdiri dari bagian-bagian berikut:

- **Create:** Memasukkan data baru ke dalam basis data (`Event::create($data)`).
- **Read:** Menampilkan isi rekaman data pada layar (`Event::all()`, `Event::find($id)`).
- **Update:** Memperbarui parameter data lama yang sudah tersimpan (`$event->update($data)`).
- **Delete:** Menghapus data secara permanen dari tabel (`$event->delete()`).

5.3.3. Route Resource

Untuk mempercepat pembuatan sekumpulan rute URL yang berkaitan dengan satu Controller khusus pengolahan data (seperti `index`, `create`, `store`, `edit`, `update`, `show`, `destroy`), Laravel menyediakan utilitas `Route::resource()`. Melalui eksekusi satu baris sintaks di `routes/web.php`, perintah tersebut secara instan menghasilkan 7 pola Route standar URL HTTP.

5.4. Instruksi Modul

Berdasarkan dokumen pedoman produk (PRD) aplikasi AmikomEventHub, Anda bertanggung jawab untuk membangun kelengkapan fitur manajemen kegiatan (Event) pada panel Admin. Secara berurutan, Anda akan merancang fungsi Read, Create, Delete, dan diakhiri Update, di mana instruksi pemrograman akan mencakup logika *Controller* maupun kerangka antarmuka *View* (Blade).

5.4.1. Membuat Controller Admin Event

1. Masuk ke Terminal pada Visual Studio Code di proyek `laravel-app` Anda.
2. Jalankan perintah terminal (*artisan command*) yang memuat perintah pembuatan *Controller* dan menghubungkannya langsung dengan referensi *Model*:

Shell

```
php artisan make:controller Admin/EventController --model=Event
```

5.4.2. Mendaftarkan Route Resource

1. Buka berkas pengatur jalur URL aplikasi pada `routes/web.php`. 2. Gunakan fungsi pengelompokan (*grouping prefix*) tingkat atas untuk mendaftarkan rute pengelolaan ini:

PHP

```
use App\Http\Controllers\Admin\EventController as EventAdminController;
```

```
Route::prefix('admin')->name('admin.')->group(function () {  
    Route::resource('events', EventAdminController::class);  
});
```

5.4.3. Konfigurasi *Fillable* di Model (Mass Assignment)

Kita wajib menginstruksikan Model agar menentukan kolom atribut mana saja yang diizinkan untuk diisi secara massal (*Mass Assignment*).

1. Buka berkas `app/Models/Event.php`.
2. Definisikan array variabel `$fillable` yang mencakup atribut yang dapat diinput:

PHP

```
protected $fillable = [  
    'category_id', 'title', 'description', 'date',  
    'location', 'price', 'stock', 'poster_path'  
];
```

```
protected $casts = [  
    'date' => 'datetime',  
];
```

```
// Menandakan atribut: 1 Event harus terpaut pada satu wujud Kategori
```

```
public function category()

{

    return $this->belongsTo(Category::class);

}
```

5.4.4. Implementasi Read - Menampilkan Daftar Event

Tahap pertama penyusunan tabel dimulai dengan membaca data dari basis data dan merendernya pada sebuah halaman antarmuka.

1. Buka berkas `app/Http/Controllers/Admin/EventController.php` dan modifikasi metode `index()`:

PHP

```
public function index()

{

    // Memakai relasi dan pengaturan limit paginasi (10 entri per halaman)

    $events = \App\Models\Event::with('category')->latest()->paginate(10);

    return view('admin.events.index', compact('events'));

}
```

2. Modifikasi `resources/views/layouts/admin.blade.php` menjadi

HTML

```
<!DOCTYPE html>

<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard - AmikomEventHub</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;50
0;600;700&display=swap"
rel="stylesheet">
  <style>
    body {
      font-family: 'Plus Jakarta Sans', sans-serif;
    }
  </style>
</head>

<body class="bg-slate-50 text-slate-900 flex min-h-screen">
  <!-- Sidebar -->
  <aside class="w-64 bg-indigo-900 text-indigo-100 flex flex-col p-6 space-y-8
sticky top-0 h-screen">
    <div class="flex items-center gap-3">
      <div class="w-10 h-10 bg-white rounded-xl flex items-center
justify-center text-indigo-900 font-bold text-xl">AH</div>
      <span class="text-xl font-bold text-white
tracking-tight">AmikomEventHub</span>
    </div>

    <nav class="flex-1 space-y-2">
      <p class="text-[10px] font-bold uppercase tracking-widest
text-indigo-400 mb-4 px-2">Main Menu</p>
      <a href="{{ route('admin.dashboard') }}" class="flex items-center
gap-3 px-4 py-3 {{ request()->routeIs('admin.dashboard') ? 'bg-indigo-800
text-white' : 'hover:bg-indigo-800' }}" rounded-xl font-bold transition">

```

```

<svg class="w-5 h-5 {{ request()->routeIs('admin.dashboard') ?
'text-indigo-300' : 'text-indigo-400' }}" fill="none" stroke="currentColor"
viewBox="0 0 24 24">

    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M4 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0
01-2-2V6zM14 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V6zM
16a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0 01-2-2v-2zM14 16a2 2 0
012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2v-2z"></path>

</svg>
Dashboard

</a>

<a href="{{ route('admin.events.index') }}" class="flex items-center gap-3 px-4
py-3 {{ request()->routeIs('admin.events.*') ? 'bg-indigo-800 text-white' :
'hover:bg-indigo-800' }}" rounded-xl font-bold transition"> <svg class="w-5 h-5
{{ request()->routeIs('admin.events.*') ?
'text-indigo-300' : 'text-indigo-400' }}" fill="none" stroke="currentColor"
viewBox="0 0 24 24">

    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2
2 0 00-2 2v12a2 2 0 002 2z"></path>

</svg>
Kelola Event

</a>

<a href="{{ route('admin.transactions.index') }}" class="flex
items-center gap-3 px-4 py-3 {{ request()->routeIs('admin.transactions.*') ?
'bg-indigo-800 text-white' : 'hover:bg-indigo-800' }}" rounded-xl font-bold
transition">

    <svg class="w-5 h-5 {{
request()->routeIs('admin.transactisons.*') ? 'text-indigo-300' :
'text-indigo-400' }}" fill="none" stroke="currentColor" viewBox="0 0 24 24">

        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M9 19v-6a2 2 0 00-2-2H5a2 2 0 00-2 2v6a2 2 0 002 2h2a2 2 0
002-2zm0 0V9a2 2 0 012-2h2a2 2 0 012 2v10m-6 0a2 2 0 002 2h2a2 2 0 002-2m0 0V5a2
2 0 012-2h2a2 2 0 012 2v14a2 2 0 01-2 2h-2a2 2 0 01-2-2z"></path>

```

```

        </svg>
        Laporan Transaksi
    </a>
</nav>

<div class="pt-6 border-t border-indigo-800">
    <form action="#" method="POST">
        @csrf
        <button type="submit" class="w-full flex items-center gap-3 px-4 py-3
        text-indigo-300 hover:text-white transition font-medium text-left"> <svg
            class="w-5 h-5" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M17 16l4-4m0 0l-4-4m4 4H7m6 4v1a3 3 0 01-3 3H6a3 3 0
01-3-3V7a3 3 0 013-3h4a3 3 0 013 3v1"></path>
            </svg>
            Keluar
        </button>
    </form>
</div>
</aside>

<!-- Main Content -->
<main class="flex-1 p-10 overflow-y-auto w-full">
    <!-- Header -->
    <header class="flex justify-between items-center mb-10 w-full
col-span-full">
        <div>
            <h1 class="text-3xl font-black">@yield('page_title',
'Dashboard')</h1>
            <p class="text-slate-500 font-medium">@yield('page_subtitle',
'Selamat datang kembali, Admin!')</p>
        </div>
        <div class="flex items-center gap-4">

```

```

        <div class="text-right hidden md:block">
            <p class="font-bold">Admin</p>
            <p class="text-xs text-slate-400">Penyelenggara Utama</p>
        </div>
        <div class="w-12 h-12 bg-white rounded-2xl shadow-sm border flex
items-center justify-center p-1">
                                                    
        </div>
    </div>
</header>

@if(session('success'))
    <div class="bg-green-100 text-green-700 p-4 rounded-xl mb-6
font-bold text-sm">
        {{ session('success') }}
    </div>
@endif

@yield('content')
</main>
</body>
</html>

```

3. Buat satu berkas *View HTML* berekstensi khusus Blade di **resources/views/admin/events/index.blade.php**, dan rakit struktur tabel *Tailwind CSS* berikut untuk melakukan perulangan (*looping*) baris data:

```

HTML
@extends('layouts.admin')

@section('title', 'Kelola Event - Admin')

```



```

@section('page_title', 'Kelola Event')

@section('page_subtitle', 'Buat dan atur acara seru Anda di sini.')

@section('content')

<div class="mb-4 text-right">

    <a href="{{ route('admin.events.create') }}" class="inline-block px-6 py-3
bg-indigo-600 text-white rounded-2xl font-bold shadow-lg shadow-indigo-100
hover:bg-indigo-700 active:scale-95 transition">

        + Tambah Event Baru

    </a>

</div>

<div class="bg-white rounded-[2.5rem] border border-slate-100 shadow-sm
overflow-hidden">

    <div class="overflow-x-auto">

        <table class="w-full text-left border-collapse">

            <thead class="bg-slate-50 text-slate-400 uppercase text-[10px]
font-black tracking-widest">

                <tr>

                    <th class="px-8 py-4 w-16">No</th>

                    <th class="px-8 py-4">Poster</th>

                    <th class="px-8 py-4">Event</th>

                    <th class="px-8 py-4">Harga / Stok</th>

                    <th class="px-8 py-4">Aksi</th>

                </tr>

```

```

</thead>

<tbody class="divide-y border-t">

    @forelse($sevents as $index => $event)

        <tr class="hover:bg-slate-50/50 transition">

            <td class="px-8 py-6 font-bold text-slate-400">{{
$sevents->firstItem() + $index }}</td>

            <td class="px-8 py-6">

                

            </td>

            <td class="px-8 py-6">

                <p class="font-black text-slate-800">{{ $event->title
}}</p>

                <p class="text-xs text-slate-400">{{
$event->category->name ?? '-' }} • {{ $event->date }}</p>

            </td>

            <td class="px-8 py-6">

                <p class="font-bold text-indigo-600">Rp {{
number_format($event->price, 0, ',', '.') }}</p>

                <p class="text-xs text-slate-400">Stok: {{ $event->stock
}}</p>

            </td>

            <td class="px-8 py-6">

                <div class="flex gap-2">

                    <!-- Catatan Modul: Deretan tombol fitur modifikasi
(U dan D) akan ditanamkan pada tahap berikutnya -->

```

```

        </div>

        </td>

    </tr>

    @empty

    <tr>

        <td colspan="5" class="px-8 py-10 text-center
text-slate-500">Belum ada acara yang ditambahkan.</td>

    </tr>

    @endforelse

</tbody>

</table>

</div>

<div class="px-8 py-6 bg-slate-50/50 border-t items-center">

    {{ $events->links() }}

</div>

</div>

@endsection

```

5.4.5. Implementasi Create - Menambah Event Baru

Alur ini menguji proses penyimpanan input masukan baru dari pengguna (*Form View*) yang ditangkap oleh *Controller*.

1. Masih di dalam berkas `EventController.php`, persiapkan rujukan referensi asal data kategori di fungsi `create()`, dan tulis logika simpan pada fungsi `store()`:

PHP

```
public function create()
{
    $categories = \App\Models\Category::all();

    return view('admin.events.create', compact('categories'));
}

public function store(\Illuminate\Http\Request $request)
{
    // Menerapkan validasi data request dari pengguna

    $data = $request->validate([
        'category_id' => 'required',
        'title' => 'required|string|max:255',
        'description' => 'required|string',
        'date' => 'required|date',
        'location' => 'required|string|max:255',
        'price' => 'required|numeric',
        'stock' => 'required|numeric'
    ]);

    // Menyimpan data yang telah divalidasi ke dalam tabel menggunakan Model

    \App\Models\Event::create($data);
}
```

```

        return redirect()->route('admin.events.index')->with('success', 'Data Event
berhasil ditambahkan.');
```

```

}
```

2. Buat berkas halaman pendataan baru letak posisi

`resources/views/admin/events/create.blade.php:`

HTML

```

@extends('layouts.admin')

@section('title', 'Tambah Event Baru - Admin')

@section('page_title', 'Tambah Event Baru')

@section('page_subtitle', 'Masukkan detail acara baru yang akan
diselenggarakan.')
```

```

@section('content')

<div class="bg-white p-8 rounded-[2.5rem] border border-slate-100 shadow-sm
max-w-3xl">

    <form action="{{ route('admin.events.store') }}" method="POST"
    enctype="multipart/form-data" class="space-y-6">

        @csrf

        <div>

            <label class="block text-sm font-bold text-slate-700 mb-2 uppercase
            tracking-wide">Judul Event</label>

            <input type="text" name="title" value="{{ old('title') }}"
            class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl Modul
```

```
focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none
transition font-medium" required>

        @error('title') <span class="text-red-500 text-sm mt-1">{{ $message
    }}</span> @enderror

    </div>

    <div>

        <label class="block text-sm font-bold text-slate-700 mb-2 uppercase
tracking-wide">Kategori</label>

        <select name="category_id" class="w-full px-5 py-4 bg-slate-50
border-2 border-slate-100 rounded-2xl focus:ring-4 focus:ring-indigo-500/10
focus:border-indigo-600 outline-none transition font-medium" required>

            <option value="">Pilih Kategori</option>

            @foreach($categories as $category)

                <option value="{{ $category->id }}" {{ old('category_id') ==
$category->id ? 'selected' : '' }}>{{ $category->name }}</option>

            @endforeach

        </select>

        @error('category_id') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror

    </div>

    <div>

        <label class="block text-sm font-bold text-slate-700 mb-2 uppercase
tracking-wide">Deskripsi</label>
```

```
        <textarea name="description" rows="4" class="w-full px-5 py-4
bg-slate-50 border-2 border-slate-100 rounded-2xl focus:ring-4
focus:ring-indigo-500/10 focus:border-indigo-600 outline-none transition
font-medium">{{ old('description') }}</textarea>
```

```
        @error('description') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror
```

```
    </div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
```

```
    <div>
```

```
        <label class="block text-sm font-bold text-slate-700 mb-2
uppercase tracking-wide">Tanggal & Waktu</label>
```

```
        <input type="datetime-local" name="date" value="{{ old('date')
}}" class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl
focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none
transition font-medium" required>
```

```
        @error('date') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror
```

```
    </div>
```

```
    <div>
```

```
        <label class="block text-sm font-bold text-slate-700 mb-2
uppercase tracking-wide">Lokasi</label>
```

```
        <input type="text" name="location" value="{{ old('location') }}"
class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl
focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none
transition font-medium" required>
```

```
        @error('location') <span class="text-red-500 text-sm mt-1">{{
```

```
$message }}</span> @enderror
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 45 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```
</div>
```

```
</div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
```

```
<div>
```

```
<label class="block text-sm font-bold text-slate-700 mb-2  
uppercase tracking-wide">Harga (Rp)</label>
```

```
<input type="number" name="price" value="{{ old('price', 0) }}"  
class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl  
focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none  
transition font-medium" required min="0">
```

```
@error('price') <span class="text-red-500 text-sm mt-1">{{  
$message }}</span> @enderror
```

```
</div>
```

```
<div>
```

```
<label class="block text-sm font-bold text-slate-700 mb-2  
uppercase tracking-wide">Kapasitas (Stok)</label>
```

```
<input type="number" name="stock" value="{{ old('stock', 1) }}"  
class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl  
focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none  
transition font-medium" required min="1">
```

```
@error('stock') <span class="text-red-500 text-sm mt-1">{{  
$message }}</span> @enderror
```

```
</div>
```

```
</div>
```



```
<div>
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 46 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```
<label class="block text-sm font-bold text-slate-700 mb-2 uppercase
tracking-wide">Poster Event (Optional)</label>
```

```
<input type="file" name="poster" accept="image/*" class="w-full px-5
py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl focus:ring-4
focus:ring-indigo-500/10 focus:border-indigo-600 outline-none transition
font-medium">
```

```
@error('poster') <span class="text-red-500 text-sm mt-1">{{ $message
}}</span> @enderror
```

```
</div>
```

```
<div class="pt-4 flex justify-end gap-4 border-t border-slate-100">
```

```
<a href="{{ route('admin.events.index') }}" class="px-6 py-4
text-slate-500 font-bold hover:text-slate-800 transition">Batal</a>
```

```
<button type="submit" class="px-8 py-4 bg-indigo-600 text-white
rounded-2xl font-bold shadow-lg shadow-indigo-100 hover:bg-indigo-700
transition">Simpan Event</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
@endsection
```

5.4.6. Implementasi Delete - Menghapus Event

Operasi penghapusan data harus menggunakan formulir tersendiri untuk mencegah manipulasi data melalui rute tautan biasa (GET). Hal ini diwajibkan untuk mengamankan proses

penghapusan data dengan *Directive* metode DELETE.

1. Kelola operasi spesifik `destroy()` pada file `EventController.php`: Modul

Praktikum : [Digital Bisnis [SI148]] Hal. 47 dari 67

```

    {
        $event->delete();
    }
}

PHP
public function destroy(Event $event)
{
    return redirect()->route('admin.events.index')->with('success', 'Data event
    berhasil dihapus secara permanen.');
```

2. Buka kembali berkas `resources/views/admin/events/index.blade.php`.

Pada area `<td class="p-4 flex gap-2">` di dalam struktur iterasi tabel, tambahkan kode tombol hapus berbalut form beserta direktif keamanan penghapusan data sebagai berikut:

```
HTML
<form action="{{ route('admin.events.destroy',
    $event->id) }}" method="POST" onsubmit="return confirm('Apakah Anda yakin ingin
    menghapus acara ini?');">
    @csrf
    @method('DELETE')
    <button type="submit" class="p-2.5
```

```

bg-rose-50 text-rose-600 rounded-xl hover:bg-rose-600 hover:text-white
transition">

<svg class="w-5 h-5" fill="none"
stroke="currentColor" viewBox="0 0 24 24">

    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0
01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1
1v3M4 7h16"></path>

```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 48 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

</svg>

</button>

</form>

```

5.4.7. Implementasi Update - Mengubah Spesifikasi Event

Tahap terakhir adalah implementasi fungsi perubahan (*Update*), yang akan menampilkan form berisi data lama dan menyimpan hasil pembaruan data tersebut menggunakan model.

1. Sisipkan referensi fungsi `edit()` dan instruksi `update()` dalam berkas `EventController.php`:

PHP

```

public function edit(Event $event)
{
    $categories = \App\Models\Category::all();

    return view('admin.events.edit', compact('event', 'categories'));
}

public function update(\Illuminate\Http\Request $request, Event $event)
{

```

```

$data = $request->validate([

    'category_id' => 'required',

    'title' => 'required|string|max:255',

    'description' => 'required|string',

    'date' => 'required|date',

    'location' => 'required|string|max:255',

```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 49 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

    'price' => 'required|numeric',

    'stock' => 'required|numeric'

]);

```

```

$event->update($data);

```

```

    return redirect()->route('admin.events.index')->with('success', 'Rincian
data event berhasil diperbarui.');
```

```

}

```

2. Struktur form perubahan akan sangat identik dengan form penambahan data baru. Buat berkas HTML `resources/views/admin/events/edit.blade.php`, pastikan untuk menambahkan direktif `@method('PUT')` serta atribut *value* agar input menampilkan data lama dari variabel `$event`.

HTML

```
@extends('layouts.admin')
```

```
@section('title', 'Edit Event - Admin')
```

```
@section('page_title', 'Edit Event')
```

```
@section('page_subtitle', 'Ubah detail acara.')
```

```
@section('content')
```

```
<div class="bg-white p-8 rounded-[2.5rem] border border-slate-100 shadow-sm  
max-w-3xl">
```

```
    <form action="{{ route('admin.events.update', $event->id) }}" method="POST"  
    enctype="multipart/form-data" class="space-y-6">
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 50 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```
@csrf
```

```
@method('PUT')
```

```
<div>
```

```
    <label class="block text-sm font-bold text-slate-700 mb-2 uppercase  
tracking-wide">Judul Event</label>
```

```
        <input type="text" name="title" value="{{ old('title',  
$event->title) }}" class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100  
rounded-2xl focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600  
outline-none transition font-medium" required>
```

```
        @error('title') <span class="text-red-500 text-sm mt-1">{{ $message  
}}</span> @enderror
```

```
</div>
```

```
<div>
```

```
    <label class="block text-sm font-bold text-slate-700 mb-2 uppercase  
tracking-wide">Kategori</label>
```

```
        <select name="category_id" class="w-full px-5 py-4 bg-slate-50  
border-2 border-slate-100 rounded-2xl focus:ring-4 focus:ring-indigo-500/10  
focus:border-indigo-600 outline-none transition font-medium" required>
```

```

        <option value="">Pilih Kategori</option>

        @foreach($categories as $category)

            <option value="{{ $category->id }}" {{ old('category_id',
$sevent->category_id) == $category->id ? 'selected' : '' }}>{{ $category->name
}}</option>

        @endforeach

    </select>

```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 51 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

        @error('category_id') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror

    </div>

    <div>

        <label class="block text-sm font-bold text-slate-700 mb-2 uppercase
tracking-wide">Deskripsi</label>

        <textarea name="description" rows="4" class="w-full px-5 py-4
bg-slate-50 border-2 border-slate-100 rounded-2xl focus:ring-4
focus:ring-indigo-500/10 focus:border-indigo-600 outline-none transition
font-medium">{{ old('description', $sevent->description) }}</textarea>

        @error('description') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror

    </div>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">

        <div>

            <label class="block text-sm font-bold text-slate-700 mb-2
uppercase tracking-wide">Tanggal & Waktu</label>

            <input type="datetime-local" name="date" value="{{ old('date',

```

```
$event->date->format('Y-m-d\TH:i')) }}" class="w-full px-5 py-4 bg-slate-50
border-2 border-slate-100 rounded-2xl focus:ring-4 focus:ring-indigo-500/10
focus:border-indigo-600 outline-none transition font-medium" required>
```

```
        @error('date') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror
```

```
</div>
```

```
<div>
```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 52 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```
<label class="block text-sm font-bold text-slate-700 mb-2
uppercase tracking-wide">Lokasi</label>
```

```
<input type="text" name="location" value="{{ old('location',
$event->location) }}" class="w-full px-5 py-4 bg-slate-50 border-2
border-slate-100 rounded-2xl focus:ring-4 focus:ring-indigo-500/10
focus:border-indigo-600 outline-none transition font-medium" required>
```

```
        @error('location') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror
```

```
</div>
```

```
</div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
```

```
<div>
```

```
<label class="block text-sm font-bold text-slate-700 mb-2
uppercase tracking-wide">Harga (Rp)</label>
```

```
<input type="number" name="price" value="{{ old('price',
$event->price) }}" class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100
rounded-2xl focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600
outline-none transition font-medium" required min="0">
```

```
        @error('price') <span class="text-red-500 text-sm mt-1">{{
$message }}</span> @enderror
```

```

        </div>

        <div>

            <label class="block text-sm font-bold text-slate-700 mb-2 uppercase tracking-wide">Kapasitas (Stok)</label>

            <input type="number" name="stock" value="{{ old('stock', $event->stock) }}" class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100

```

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 53 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

rounded-2xl focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none transition font-medium" required min="1">

        @error('stock') <span class="text-red-500 text-sm mt-1">{{ $message }}</span> @enderror

    </div>

</div>

<div>

    <label class="block text-sm font-bold text-slate-700 mb-2 uppercase tracking-wide">Poster Event (Opsional)</label>

    <input type="file" name="poster" accept="image/*" class="w-full px-5 py-4 bg-slate-50 border-2 border-slate-100 rounded-2xl focus:ring-4 focus:ring-indigo-500/10 focus:border-indigo-600 outline-none transition font-medium">

    @if($event->poster_path)

        <p class="text-sm text-slate-500 mt-2">Poster saat ini: <a href="{{ asset('storage/' . $event->poster_path) }}" target="_blank" class="text-indigo-600 hover:underline">Lihat</a></p>

    @endif

```



```

        @error('poster') <span class="text-red-500 text-sm mt-1">{{ $message
    }}</span> @enderror

</div>

<div class="pt-4 flex justify-end gap-4 border-t border-slate-100">

    <a href="{{ route('admin.events.index') }}" class="px-6 py-4
text-slate-500 font-bold hover:text-slate-800 transition">Batal</a> Modul Praktikum

```

: [Digital Bisnis [SI148]] Hal. 54 dari 67

Universitas AMIKOM Yogyakarta
Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

```

        <button type="submit" class="px-8 py-4 bg-indigo-600 text-white
rounded-2xl font-bold shadow-lg shadow-indigo-100 hover:bg-indigo-700
transition">Simpan Perubahan</button>

</div>

</form>

</div>

@endsection

```

3. Langkah penyelesaian: Tambahkan tautan menu penyuntingan (*Edit*) berdampingan dengan tombol hapus yang Anda buat sebelumnya di struktur baris tabel `resources/views/admin/events/index.blade.php`:

HTML

```

<a href="{{ route('admin.events.edit', $event->id) }}" class="p-2.5 bg-indigo-50
text-indigo-600 rounded-xl hover:bg-indigo-600 hover:text-white transition">

```

```

<svg class="w-5 h-5" fill="none"
stroke="currentColor" viewBox="0 0 24 24">

<path stroke-linecap="round"
stroke-linejoin="round" stroke-width="2" d="M11 5H6a2 2 0 0-2 2v11a2 2 0 0-2
2h11a2 2 0 0-2 2v-5m-1.414-9.414a2 2 0 112.828 2.828L11.828
15H9v-2.828L8.586-8.586z"></path>

</svg>

</a>

```

5.5. Pengujian dan Tugas Mandiri

Pada modul ini, pembuatan *View* tidak dibebankan sebagai tugas tambahan. Fokus praktikum ditekankan pada pengujian integrasi fungsional dari awal hingga akhir (*End-to-End Test*).

Modul Praktikum : [Digital Bisnis [SI148]] Hal. 55 dari 67

Universitas AMIKOM Yogyakarta

Fakultas Ilmu Komputer - Program Studi S1 Sistem Informasi

1. Pastikan Anda telah menamai ekstensi berkas (*file*) dengan benar dan memanggil rute (*route*) yang tepat. Jalankan perintah server virtual pada terminal (*php artisan serve*). 2. Terapkan serangkaian metode evaluasi untuk mensimulasikan operasional aplikasi: - Mulailah dengan mengakses halaman indeks Event (*Read*), pastikan formasi baris data tampil dengan benar.
 - Lakukan penambahan data Event baru melalui formulir input (*Create*). - Jalankan proses edit data (*Update*) dengan mengganti detail tiket atau harga. - Pastikan validasi pengisian form bekerja dengan tepat tanpa menyebabkan gangguan (*error*).
 - Terakhir, hapus salah satu data Event fiktif tersebut menggunakan tombol Hapus (*Delete*).
3. Dokumentasikan setiap tahap operasional aplikasi ini menggunakan bukti tangkapan layar (*screenshot*) secara beruntun (Mulai dari pengisian formulir, penunjukkan layar tabel awal, keberhasilan pembaruan data, dan bukti layar kosong setelah data dihapus).
4. Sempurnakan laporan praktikum Anda ke dalam format dokumen **PDF** guna diserahkan pada portal E-Learning maksimal sebelum pertemuan praktikum minggu berikutnya.

